# Bash

# INF20125L

Felipe Dumont Ortiz
Felipe.dumont@usm.cl

# Iniciemos lubuntu

- Utilizaremos la distribución **lubuntu** en los laboratorios
- Para ingresar en un laboratorio tienen que utilizar el usuario **.\test** y la contraseña **test**
- Abriremos dentro de esta sesión la máquina virtual de oracle, Oracle VirtualBox y **iniciaremos la máquina virtual** seleccionada lubuntu

# Bash

Recordando de la semana pasada

```
#!/bin/bash
echo "Ejecutando pruebas..."
# Caso 1
./programa < input/caso1.txt > output/out1.txt
diff output/out1.txt expected/exp1.txt
```

¿Cómo ejecutar?

```
chmod +x test.sh
./test.sh
```

**La primera línea, describe el tipo de archivo, después mostramos por la terminar el texto con echo, para luego ejecutar nuestro programa y determinar errores**

# Bash - variables

```
#!/bin/bash

nombre="Valor"
echo "$nombre"
valor=1
```

**Importante**, el valor antes o después de = tiene que estar pegado. El termino $ es pedir el valor de la variable

# Bash - variables

```
str="Hello World"  # String
num=10             #  Se trata como entero o tring, baso en uso
num=$((num+5)) # Tratado explicitamente como entero
arr=(1 2 3)        # Arreglo
```

**Las variables por lo general se consideran strings**

# Bash - variables

```
num=100
num=$num+10
echo "$num"
```

```
num=100
num=$((num+10))
echo "$num"
```

**Tienen distintos valores finales para num, uno funciona como string y el otro como entero**

# Bash - Inputs

.test.sh 10 carpeta/interna 2026

$# → Cantidad de parámetros
$* → todos los parámetros juntos
$N → parámetro n
#0 → Nombre del script

# Bash - utilizando comandos

```
#!/bin/bash
var=$(pwd)
results=$(ll)
echo "$var"
echo "$results"
```

Los resultados de la ejecución de un comando se pueden guardar en una variable, por ejemplo las llamadas pwd y ll

# Bash - utilizando comandos

```
#!/bin/bash
var=$(pwd)
results=$(ll)
echo "$var"
echo "$results"
```

Los resultados de la ejecución de un comando se pueden guardar en una variable, por ejemplo las llamadas pwd y ll

# Bash - Ciclo for

Similar a C

```
#!/bin/bash
for ((i=1;i<=5;i++))
do
    echo "$i"
done
```

# Bash - Ciclo for

Similar a C

```
#!/bin/bash
for ((i=1;i<=5;i++))
do
    echo "$i"
done
```

Con listas

```
#!/bin/bash
for i in 1 2 3 4
do
    echo "$i"
done
```

# Bash - Ciclo for

Con arreglos

```
#!/bin/bash
arr=("Pedro" "Juan" "Diego")
for x in arr
do
    echo "$i"
done
```

**Utilizar espacios de separación en el arreglo o los considerará como una sola palabra**

# Bash - Ciclo for

Con Carpetas

```
#!/bin/bash
for file in carpeta/archivo*.txt
do
   echo "$file”
done
```

**Caracteres especiales:**

***** → Calza cualquier secuencia (más de un caracter)
**?** → Calza un carácter
**[1z]** → Calza con cualquier caracter

**Utilizar espacios de separación en el arreglo o los considerará como una sola palabra**

# Bash - Condicional

```
#!/bin/bash
if [ cond ]; then
    # comandos
elif [ cond ]; then
    # comandos
else
    # comandos
fi
```

**Operadores**

-eq → igual a
-ne → distinto de
-it → menor que
-gt → mayor que

**Para strings**

= → igual a

**Utilizar espacios de separación en el arreglo o los considerará como una sola palabra**

# Bash - retorno de comando

**Todos los comandos al ser ejecutados retornan 0 en caso de exito y distinto de 0 en caso de error.**

```
#!/bin/bash
gcc programa.c -o programa
echo $?  # CUIDADO !!!!
if [ $? -ne 0 ]; then
    echo "Error"
fi
```

**Utilizar espacios de separación en el arreglo o los considerará como una sola palabra**

# Bash

# INF20125L

Felipe Dumont Ortiz
Felipe.dumont@usm.cl